Basic Security Mechanisms

Ehsan Khamespanah

Spring 2024

Problem Definition

- We talked about how to login and session manipulation but what about authentication?
- User and Password have to be sent from clients to servers
- But what about the security of the communication medium between clients and servers



Changing HTTP Method

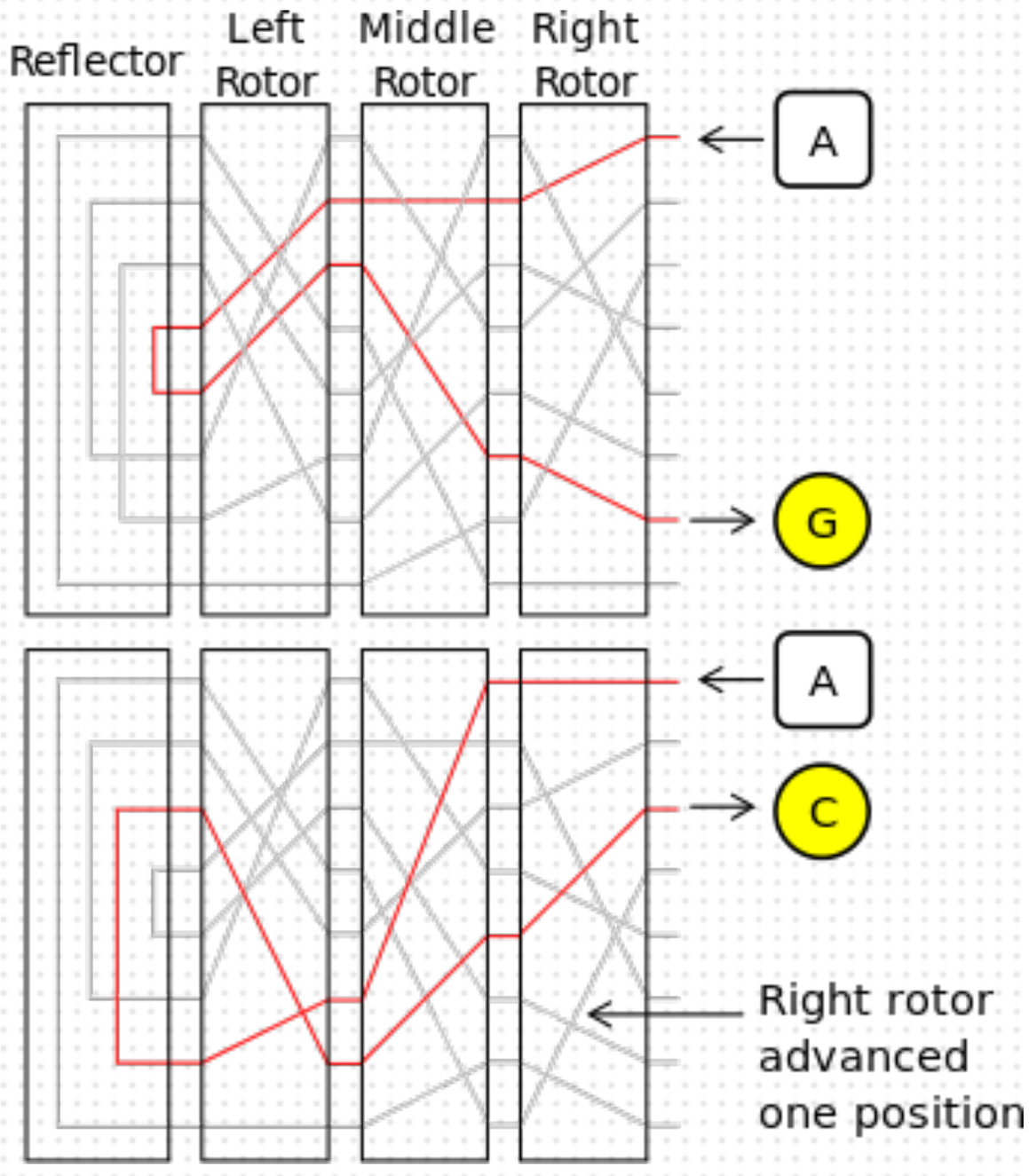
- Using GET method to send credential data: Parameters are show in address bar of browsers
  - Steal credential info with a glance!
- Using POST method to send credential data
  - Data is hidden inside HTTP packets
  - Communication medium sniffing
  - Broadcasting is a common approach for data transfer in local networks!

It Is an Old Story!

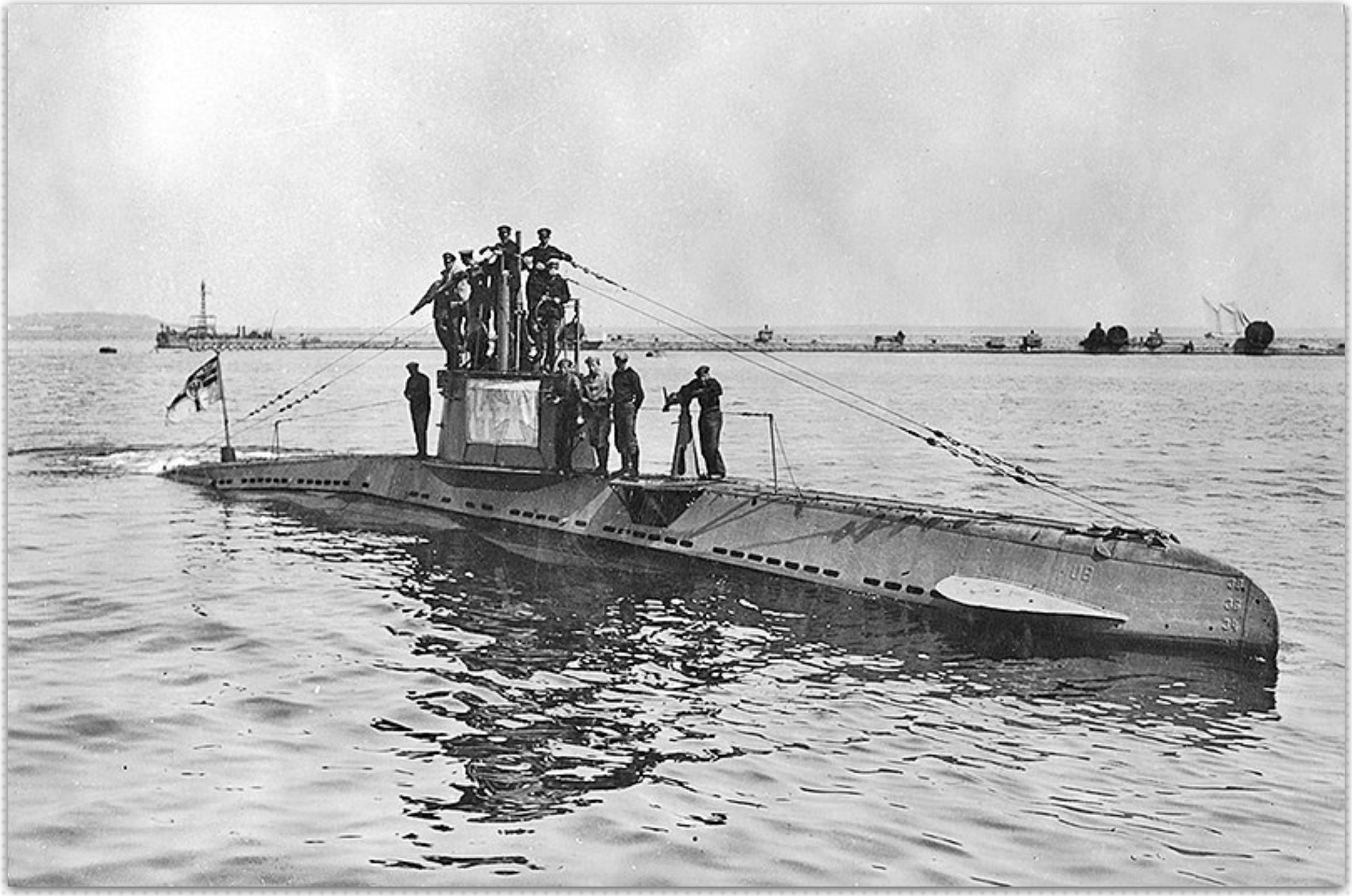
- **1587:** Ciphers from Mary of Scots plotting assassination of queen Elizabeth broken; used as evidence to convict her of treason.
- **1860's (civil war):** Confederacy used good cipher (Vigenere) in a bad way. Messages routinely broken by team of young union cryptanalysts; in particular leading to a Manhattan manufacturer of plates for printing rebel currency.
- **1878:** New York Tribune decodes telegram proving Democrats' attempt to buy an electoral vote in presidential election for $10K.
- **1914:** With aid of partial info from sunken German ships, British intelligence broke all German codes. Cracked telegram of German plan to form alliance with Mexico and conquer back territory from U.S.
- **WWII:** Cryptanalysis used by both sides. Polish & British cryptanalysts break supposedly unbreakable Enigma cipher using mix of ingenuity, German

Enigma Machine





The Story of U Boats



One Way Encryption

- There is no need to decrypt the given data in the destination
  - We need to know that the correct data is sent!
- Sending username without encryption but password with one-way encryption
  - Password in the receiver side can be encrypted using the same approach and comparing the encrypted data

MD5 Hashing

- MD5 hashing algorithm is a light-weight approach for encrypting data and sending it to the receiver
  - “frog” -> “938c2cc0dcc05f2b68c4287040cfcf71”
- Need for a random attachment (salt text)
  - “frog”+”random-text” -> “not-reusable-text”
- Using GPU to crack the hashing algorithm
  - Brute force for cracking in few days

Substitution - Caesar Cipher

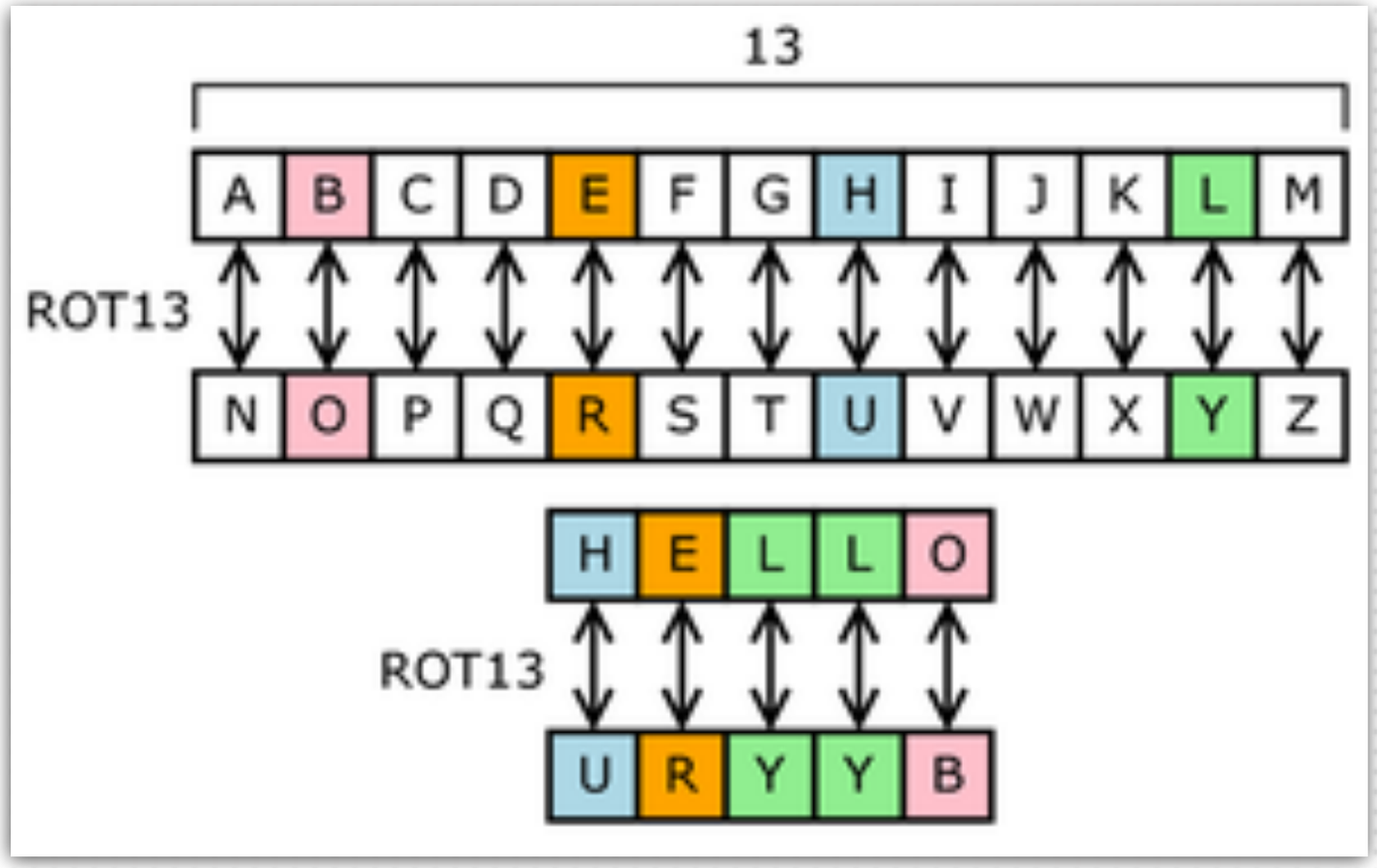
- Each letter in the plaintext is replaced by a letter some fixed number of positions down the alphabet
  - Key: k = no. between 0 and 25.
  - Encryption: encode the ith letter as the $(i+k)^{th}$
  - Decryption: decode the jth letter to the $(j-k)^{th}$

k = 2	U G P F T F K P H Q T E G O G P V
	S E N D R E I N F O R C E M E N T

- Problem: only 26 possibilities for key – can be broken in short time.

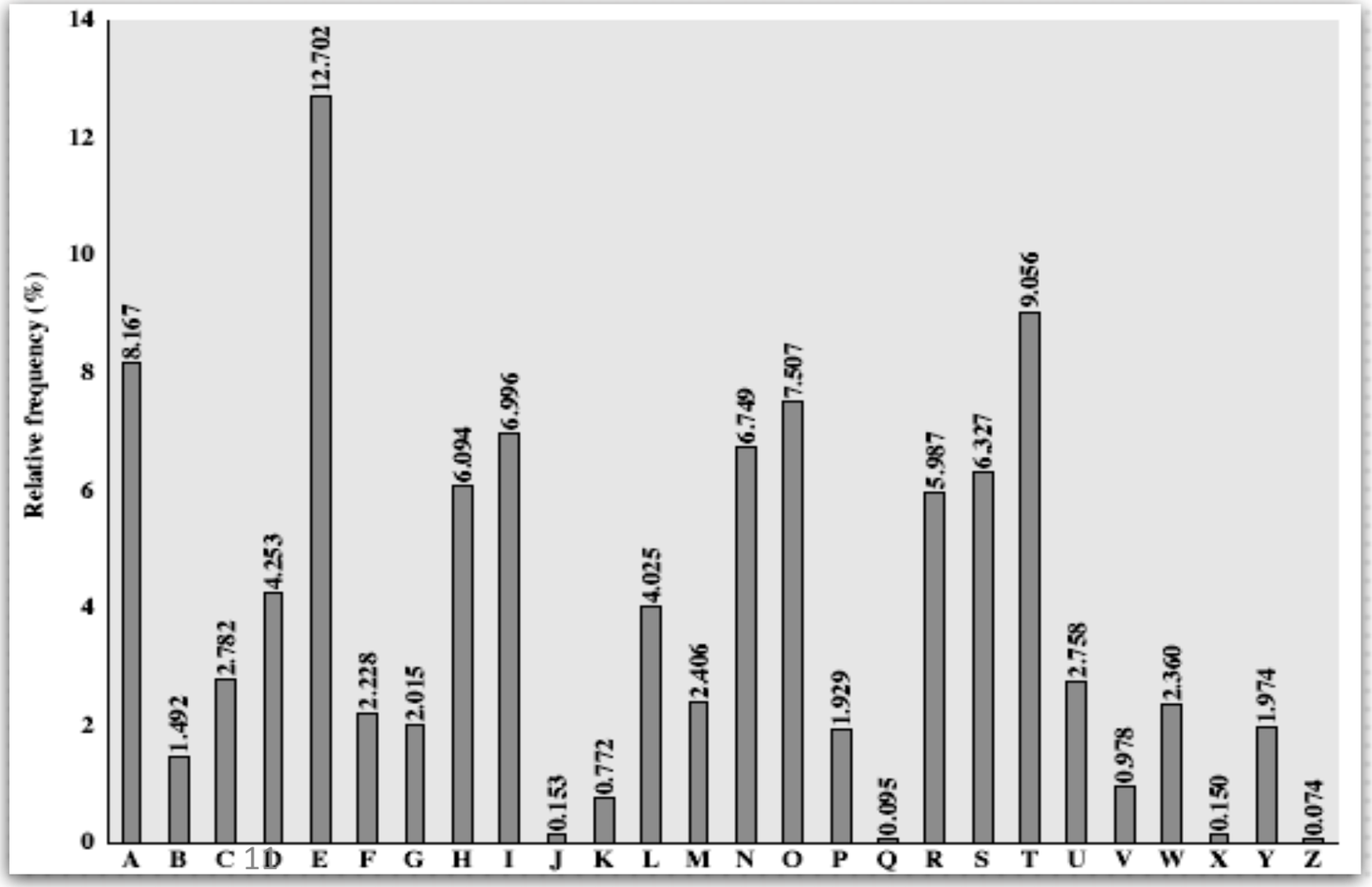
Substitution - Random Cipher

- Plaintext alphabet: ABCDEFGHIJKLMNOPQRSTUVWXYZ
- Ciphertext alphabet: NOPQRSTUVWXYZABCDEFGHIJKLM



Substitution - Random Cipher

- Number of possible keys: 26!
  (403,291,461,126,605,635,584,000,000)
- Key observation: can recover plaintext using statistics on letter frequencies.



Substitution - Vigenère Cipher

- The Vigenère cipher has several Caesar ciphers in sequence with different shift values.
- The person sending the message chooses a keyword and repeats it until it matches the length of the plaintext

Plaintext: ATTACKATDAWN

Key: LEMONLEMONL

Ciphertext: LXFOPVEFRNHR

	A	**B**	**C**	**D**	**E**	**…**	**Y**	**Z**
A	A	B	C	D	E	…	Y	Z
B	B	C	D	E	F	…	Z	A
C	C	D	E	F	G	…	A	B
D	D	E	F	G	H	…	B	C
E	E	F	G	H	I	…	C	D
…	…	…	…	…	…	…	…	…
Y	Y	Z	A	B	C	…	W	X
Z	Z	A	B	C	D	…	X	Y

Substitution - Vigenère Cipher

- The Vigenère cipher has several Caesar ciphers in sequence with different shift values.
- The person sending the message chooses a ke[illegible]word and repeats it until it matches the length of the plaintext

Plaintext: ATTAC[illegible]

Key: LE[illegible]

Ciphertext: LXFC[illegible]RNHR

Considered "unbreakable" for 300 years (broken by Babbage, Kasiski 1850's)

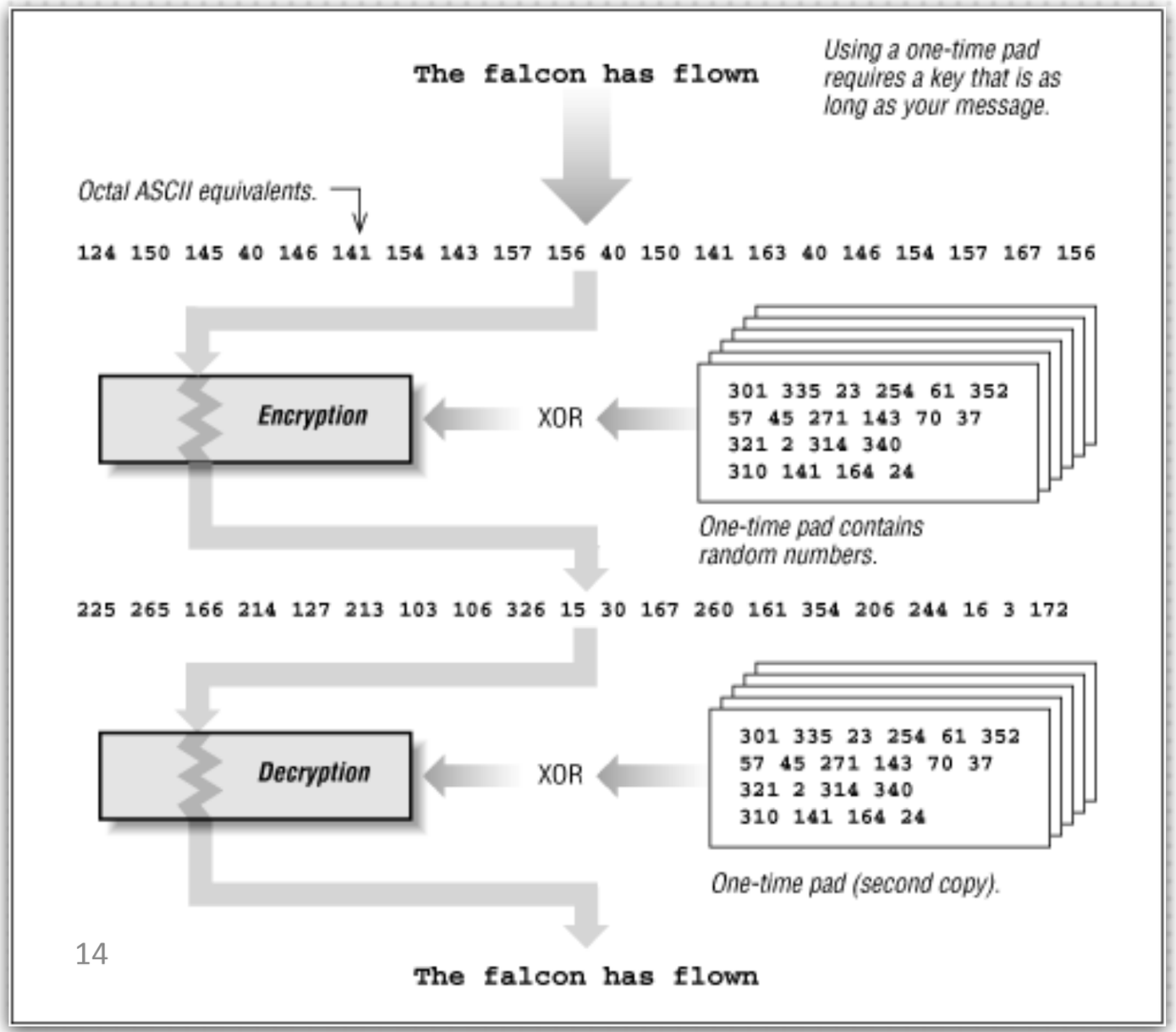
						…	Y	Z
					E	…	Y	Z
				E	F	…	Z	A
			E	F	G	…	A	B
	D	E	F	G	H	…	B	C
E	E	F	G	H	I	…	C	D
…	…	…	…	…	…	…	…	…
Y	Y	Z	A	B	C	…	W	X
Z	Z	A	B	C	D	…	X	Y

Encryption Keys

- One-time pad
  - Using XOR between data and a given key
- Symmetric key
  - A shared secret that all parties who participate must know (or the decryption key can be computed from the encryption key).
- Asymmetric key
  - Public and Private key such that public key is openly distribute to all parties and the decryption key remained secret and cannot be computed from the public key.

One-Time Pad Key (OTP)

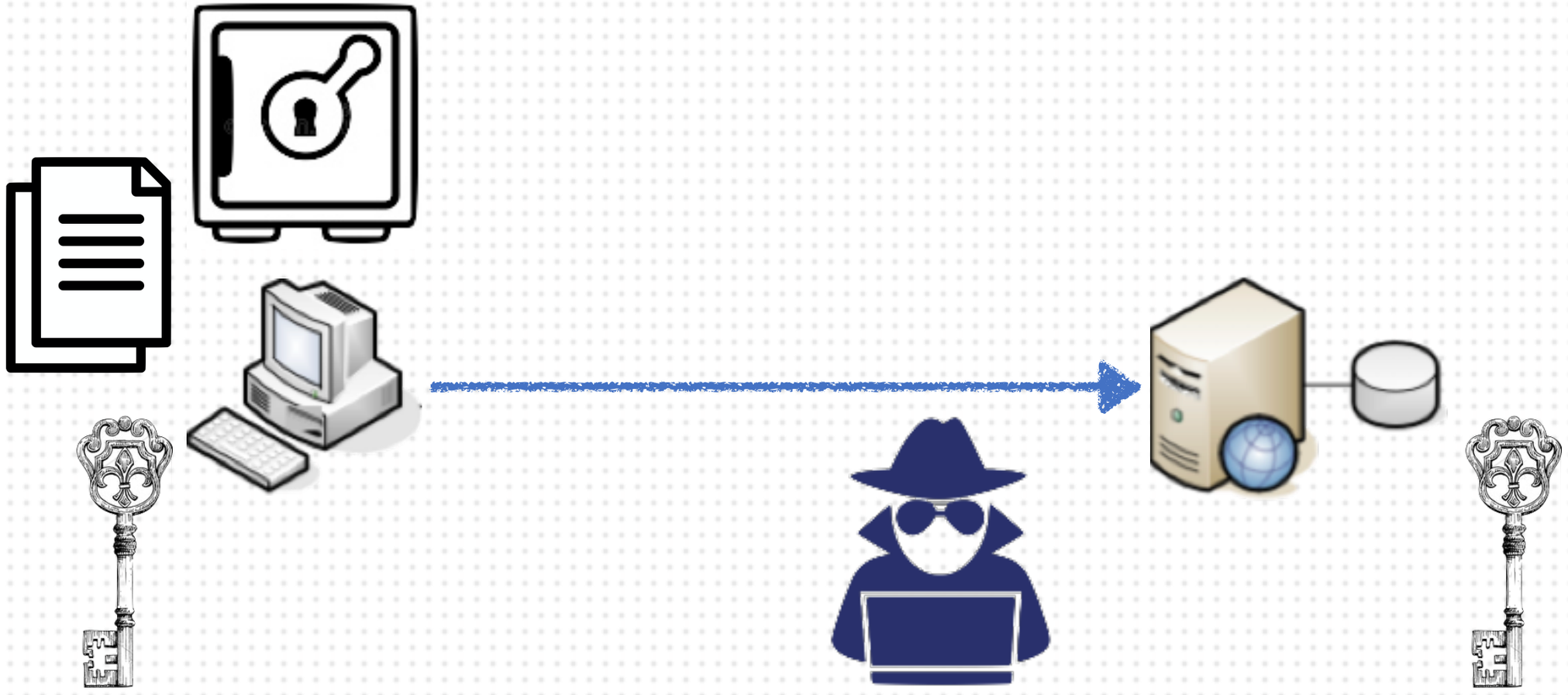
- Using one-time pre-shared key the same size as, or longer than, the message being sent
- It is impossible to decrypt or break if:
  1) truly random,
  2) as long as the plaintext
  3) never reused
  4) kept secret
- Hard to distribute keys



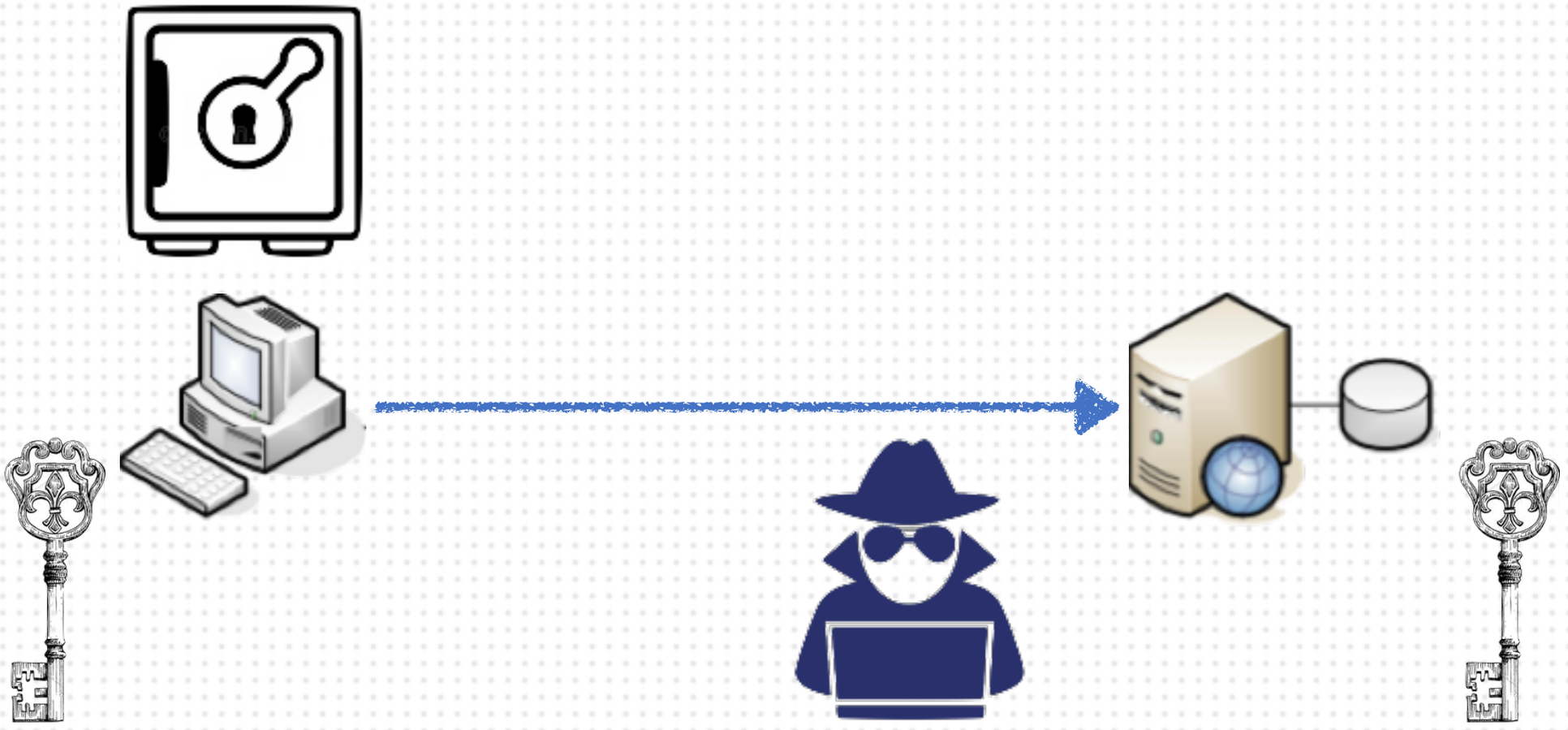
Symmetric Key



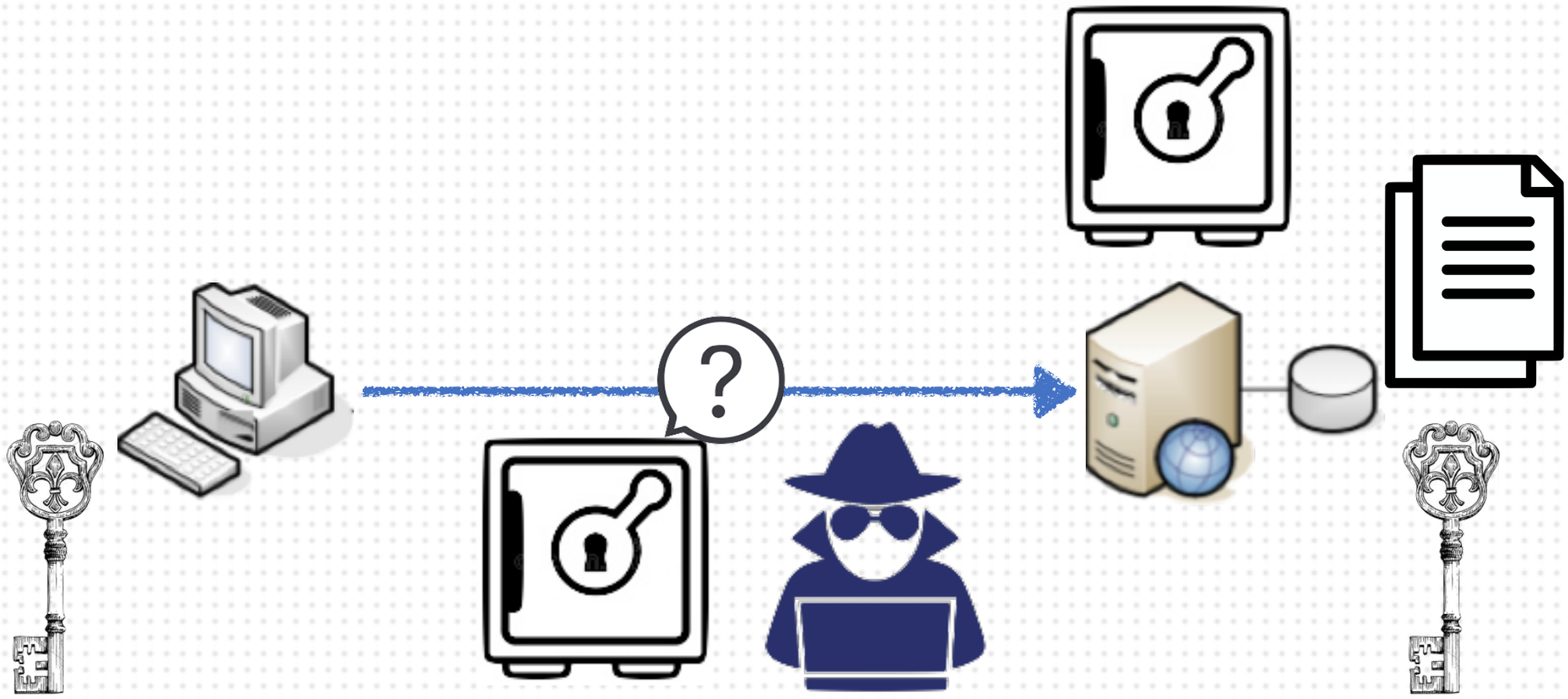
Symmetric Key



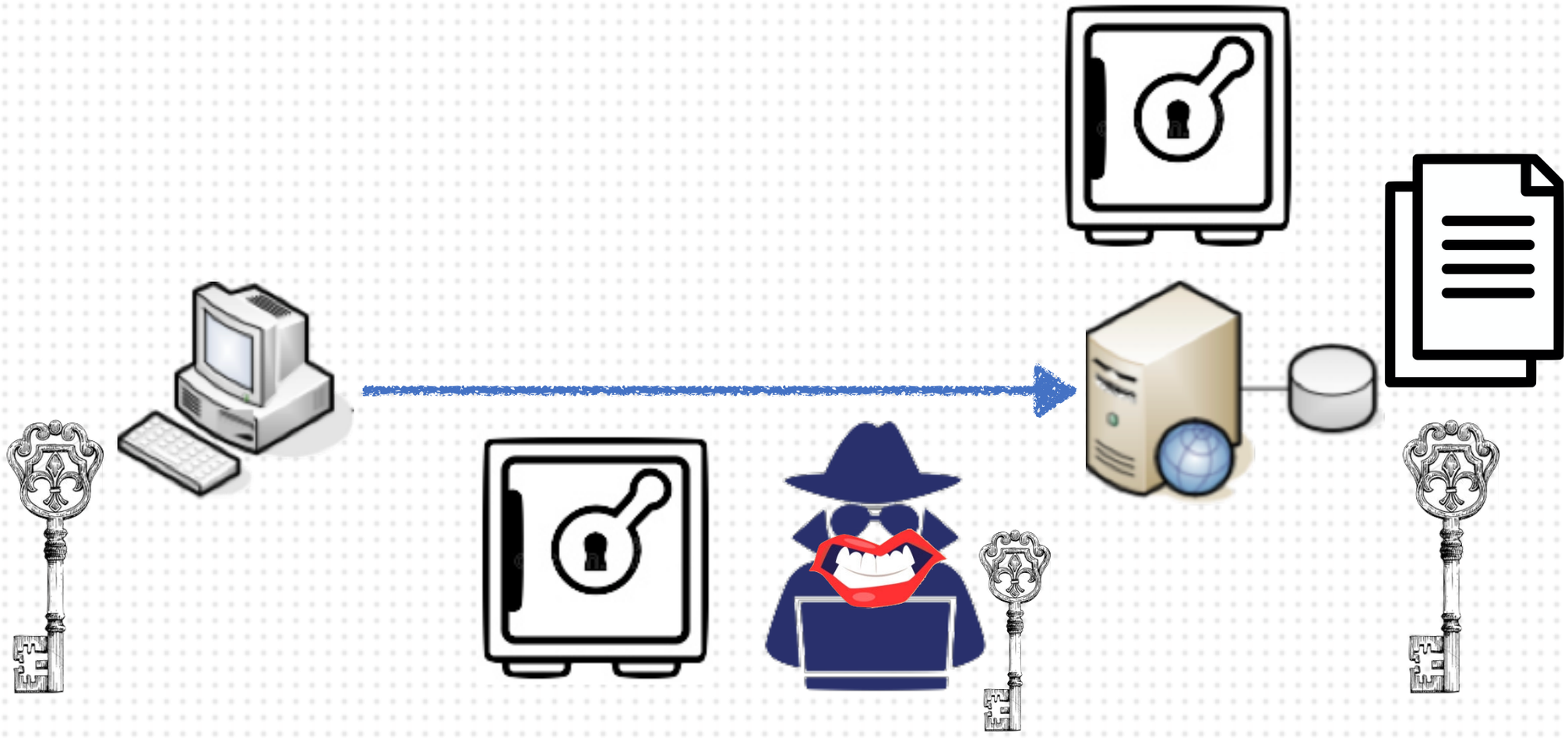
Symmetric Key



Symmetric Key



Symmetric Key



- How to send the key from a server to a client?
- Man in the middle maybe steal the key on its way!

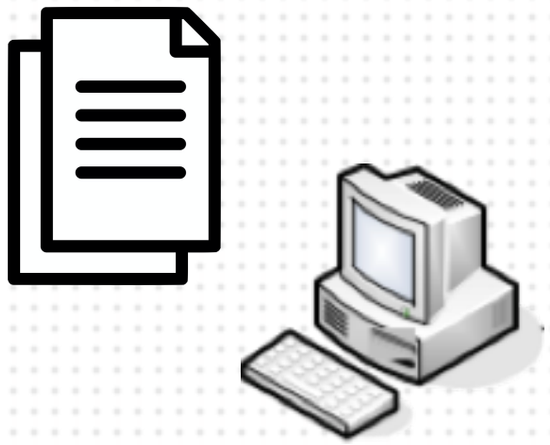
Asymmetric Key



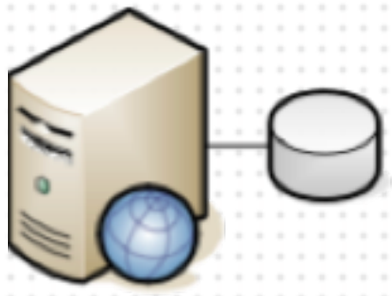
Asymmetric Key

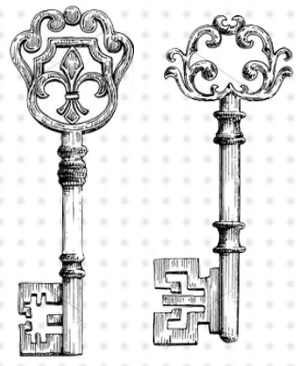


Asymmetric Key

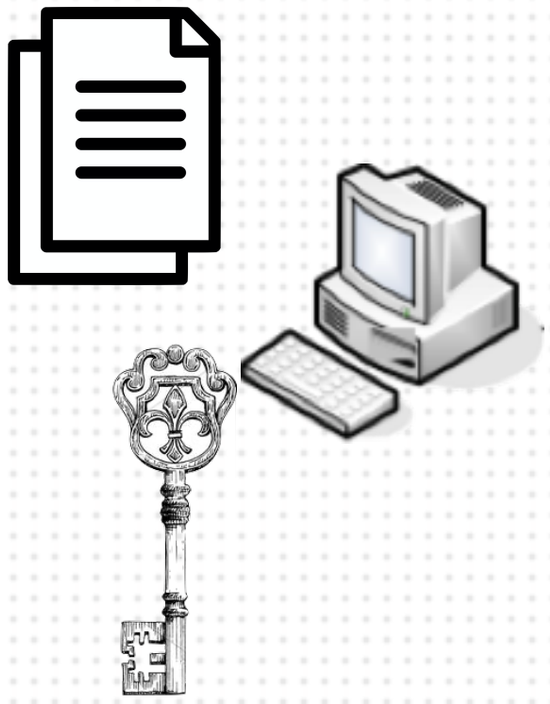


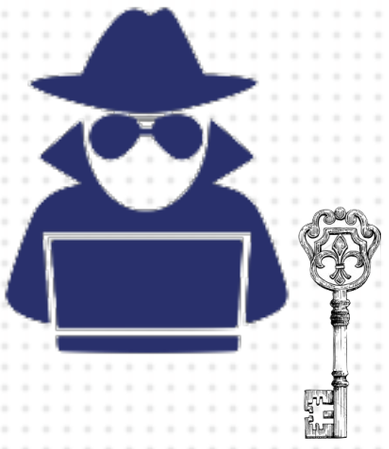


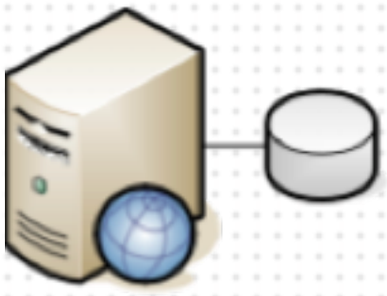


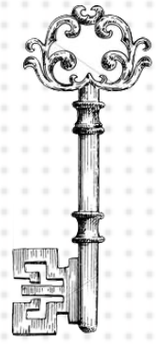


Asymmetric Key

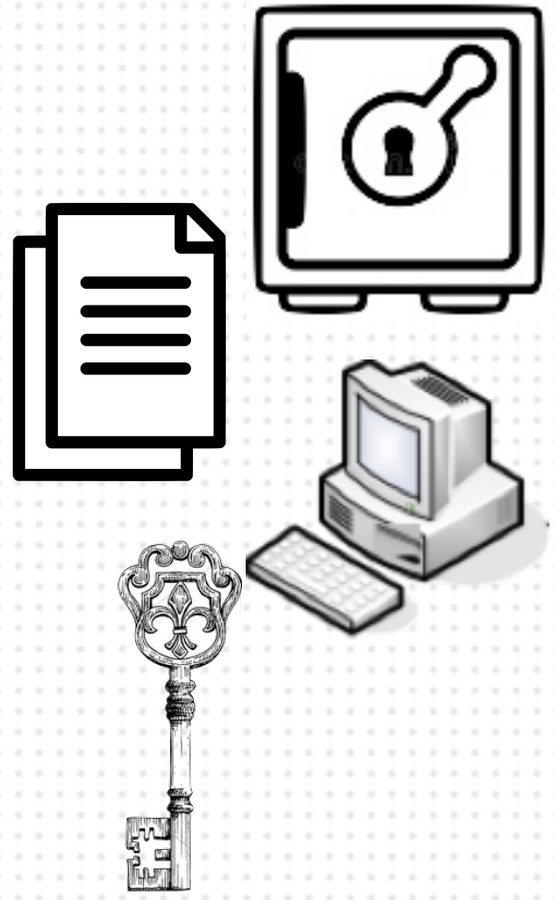




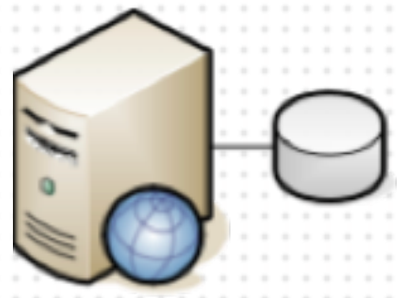


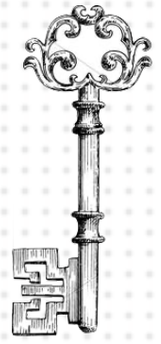


Asymmetric Key

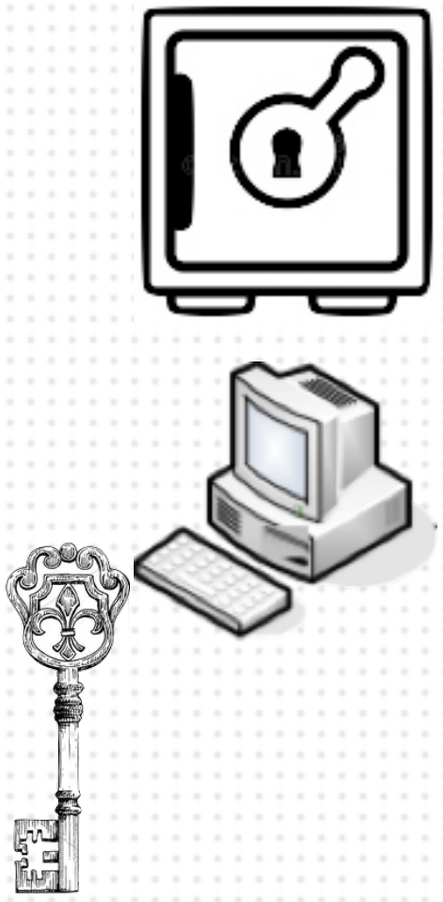




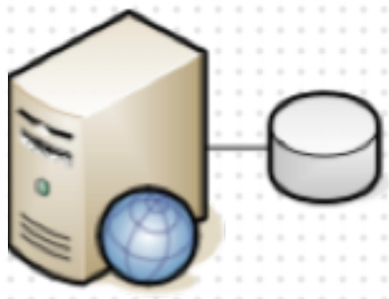


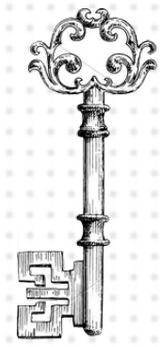


Asymmetric Key

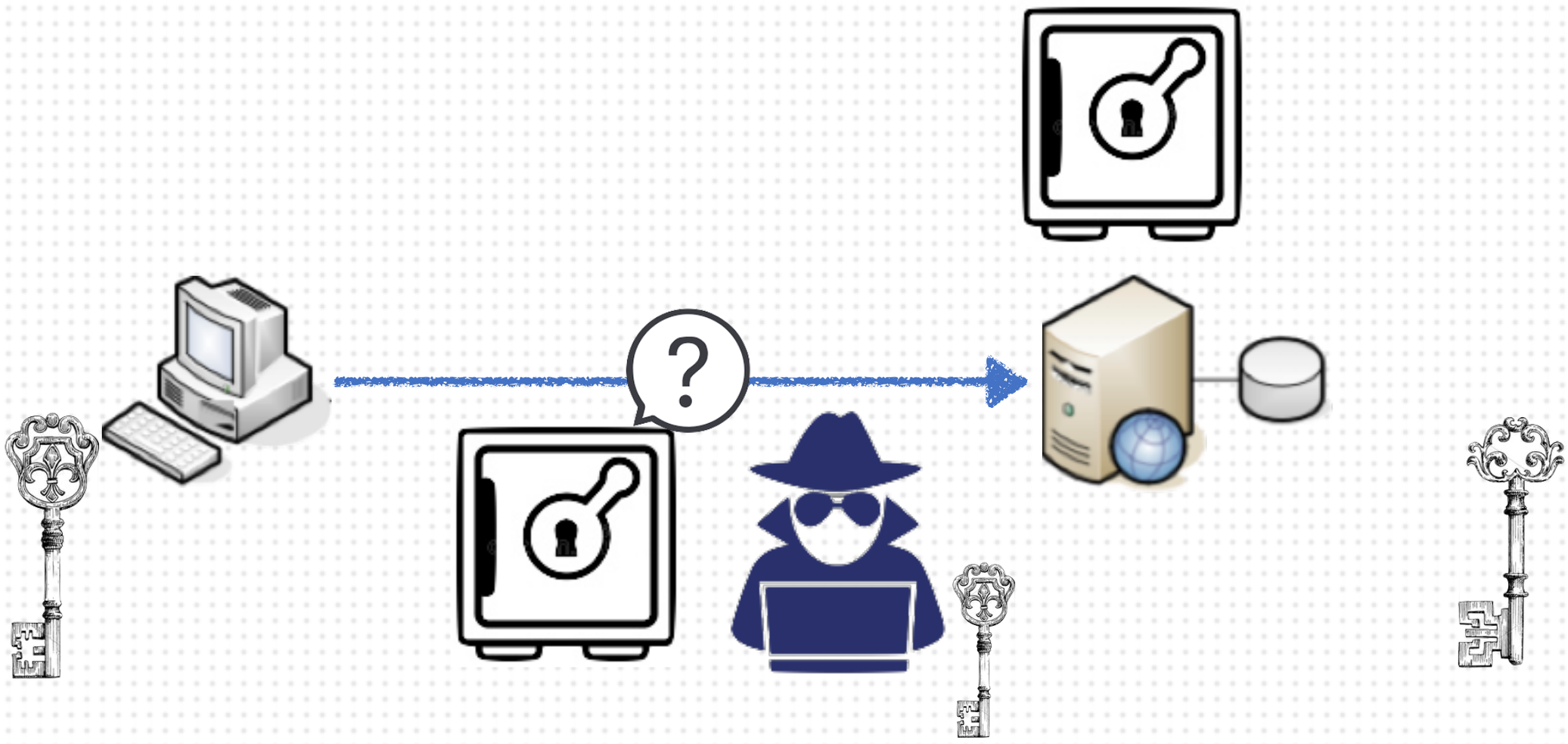




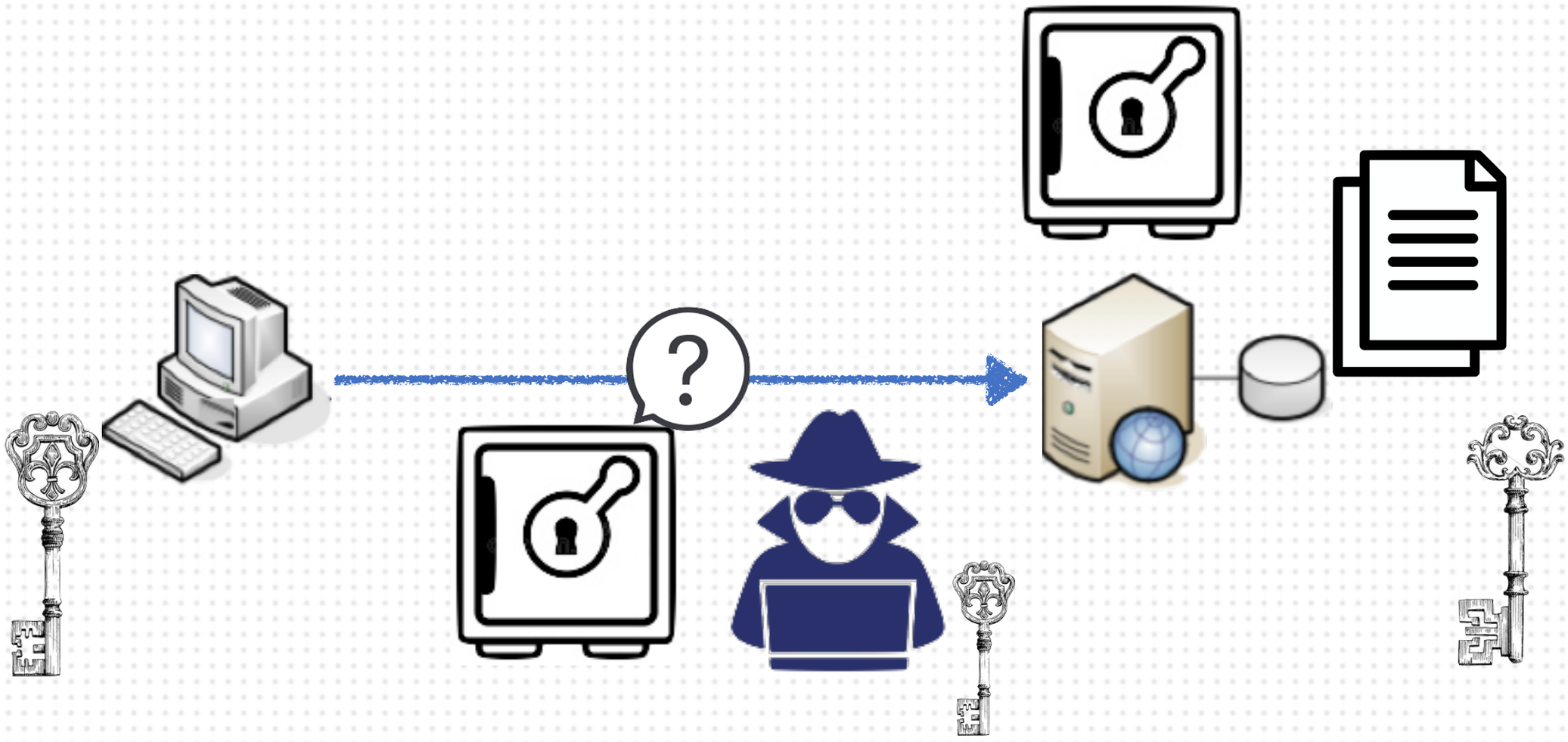




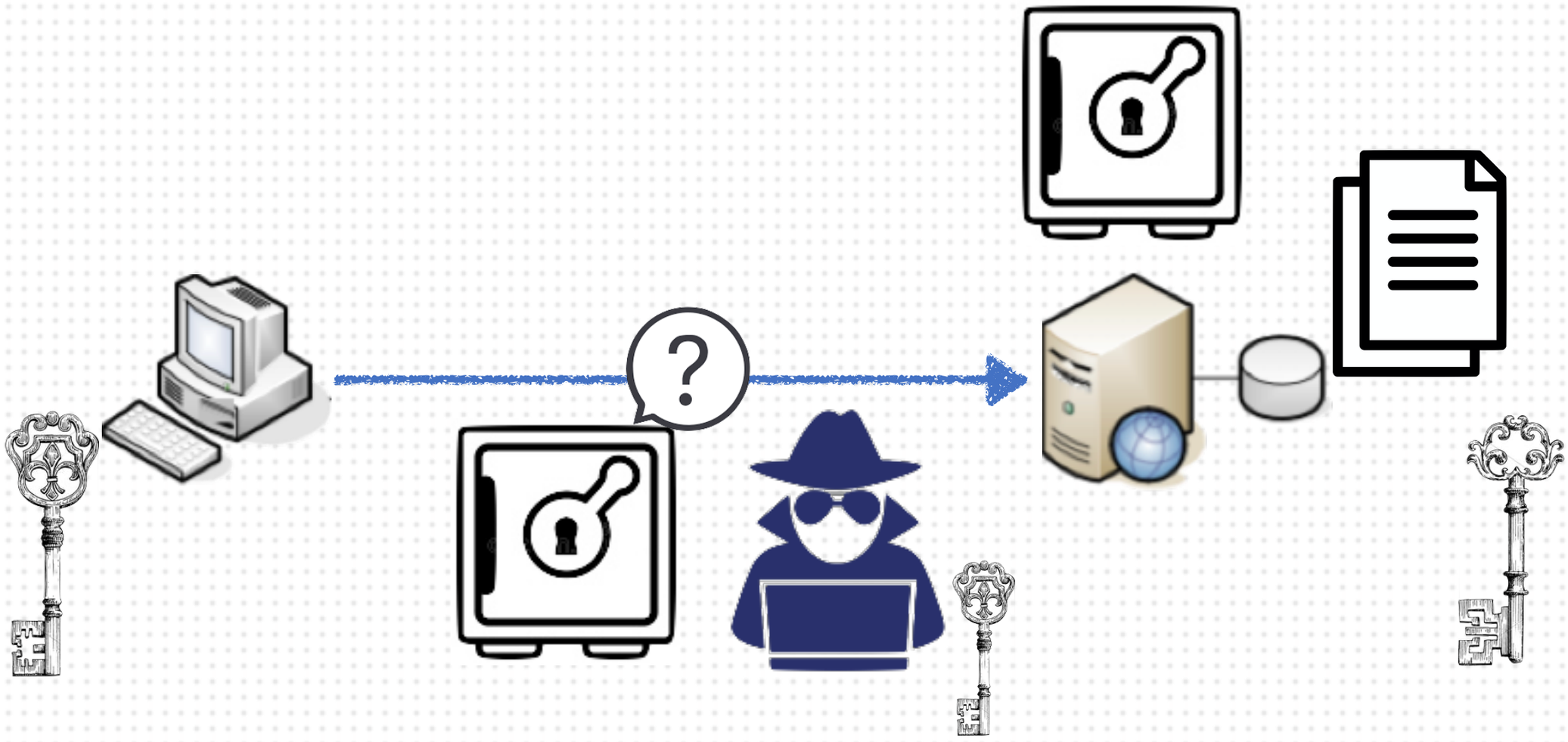
Asymmetric Key



Asymmetric Key



Asymmetric Key



- Seems to be safe!
- But very slow

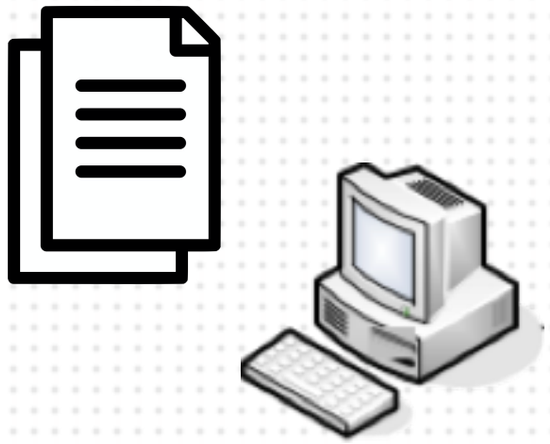
Asymmetric Key - Attack Scenario



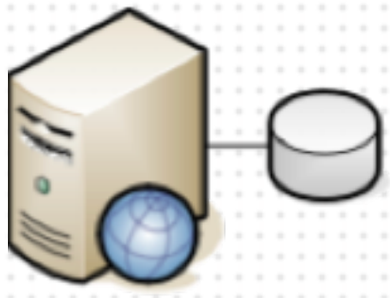
Asymmetric Key - Attack Scenario

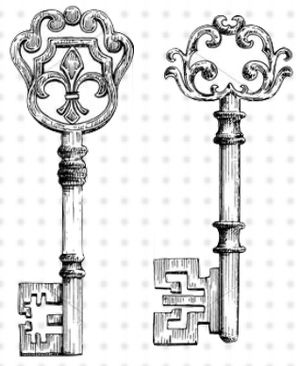


Asymmetric Key - Attack Scenario

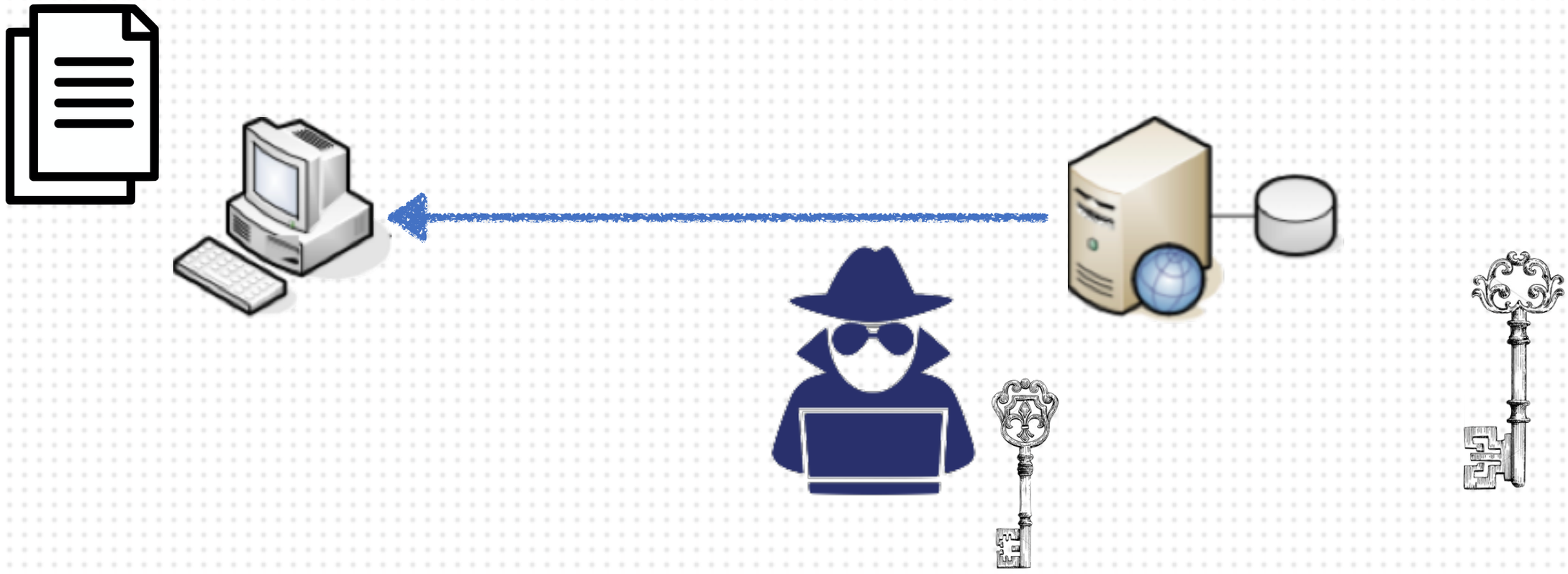




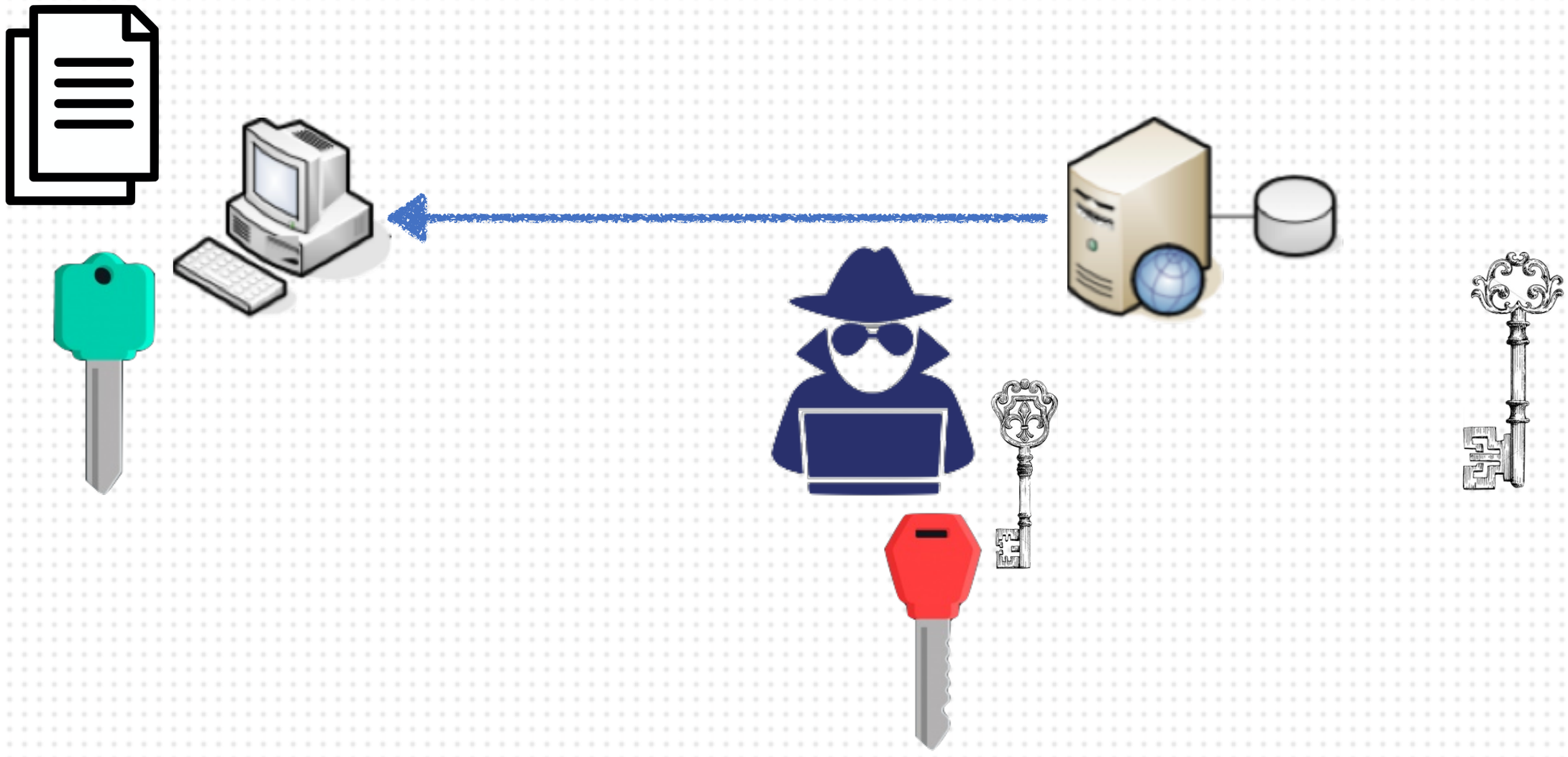




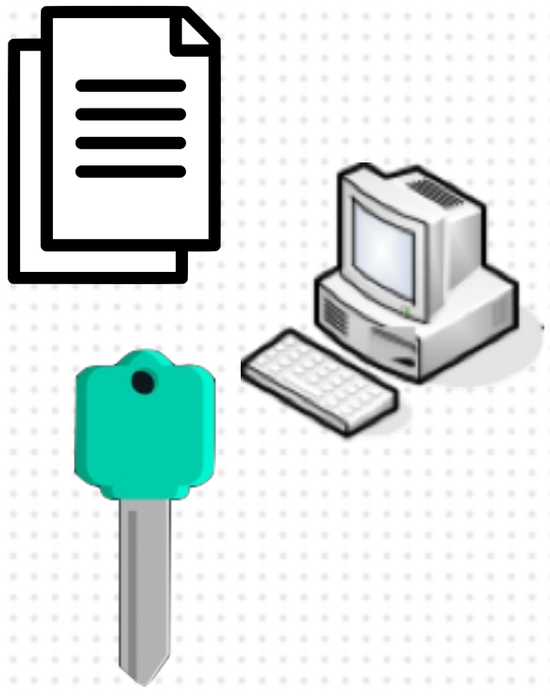
Asymmetric Key - Attack Scenario



Asymmetric Key - Attack Scenario



Asymmetric Key - Attack Scenario

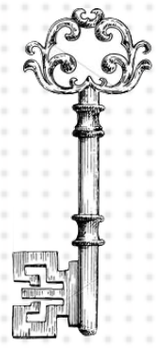




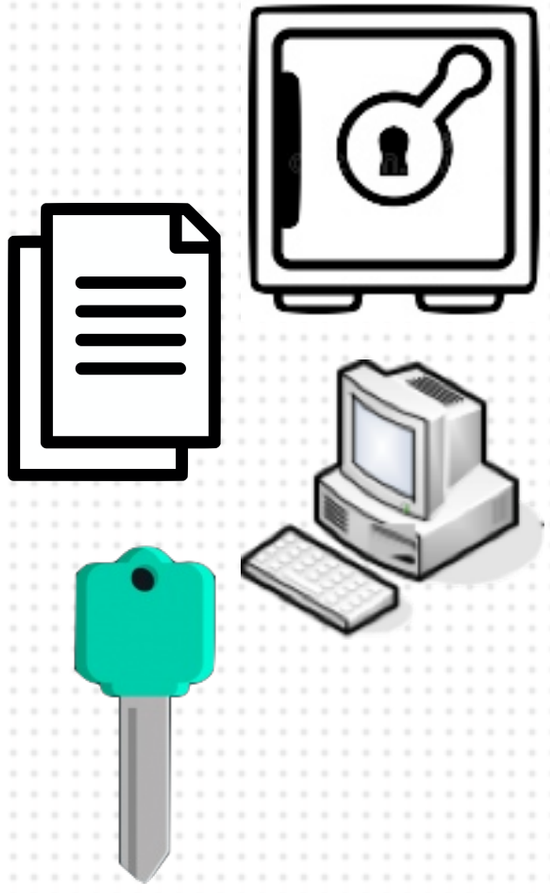


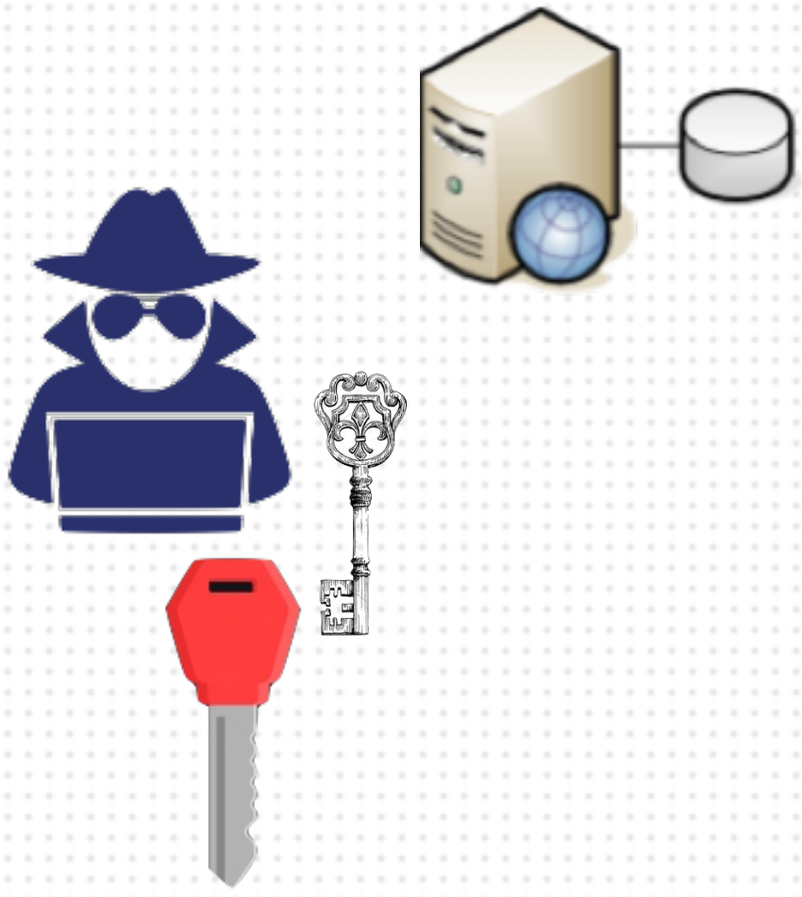




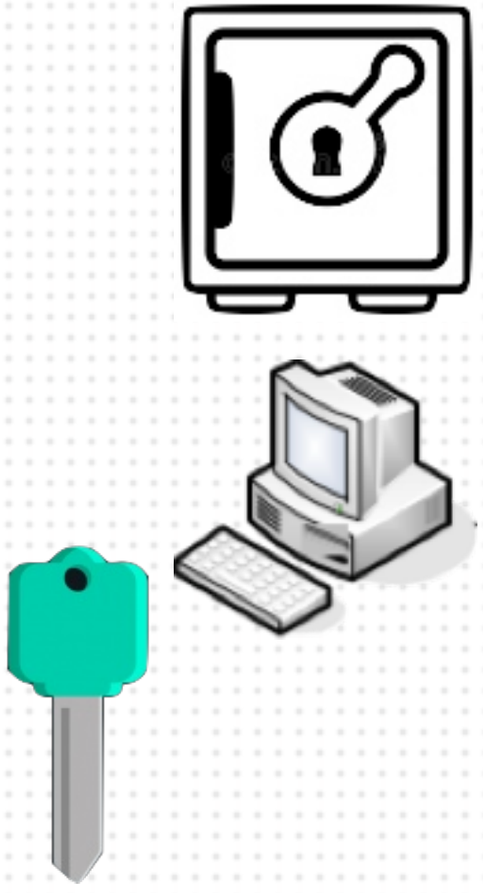


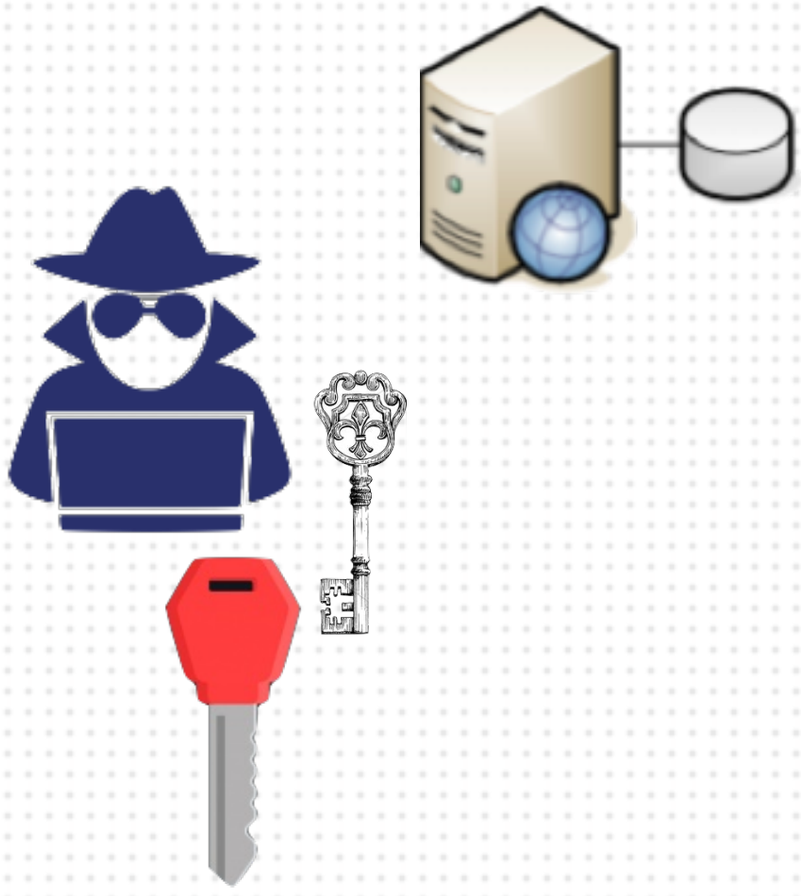
Asymmetric Key - Attack Scenario



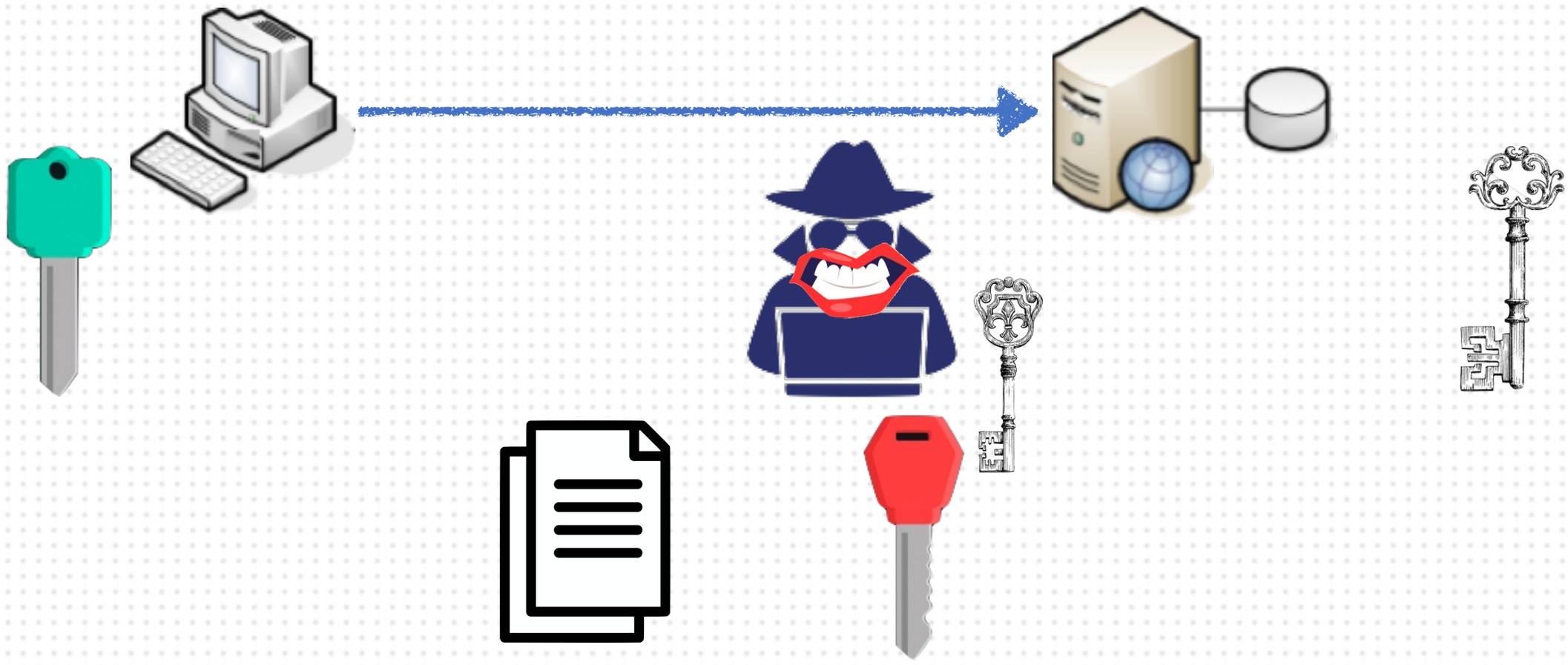


Asymmetric Key - Attack Scenario

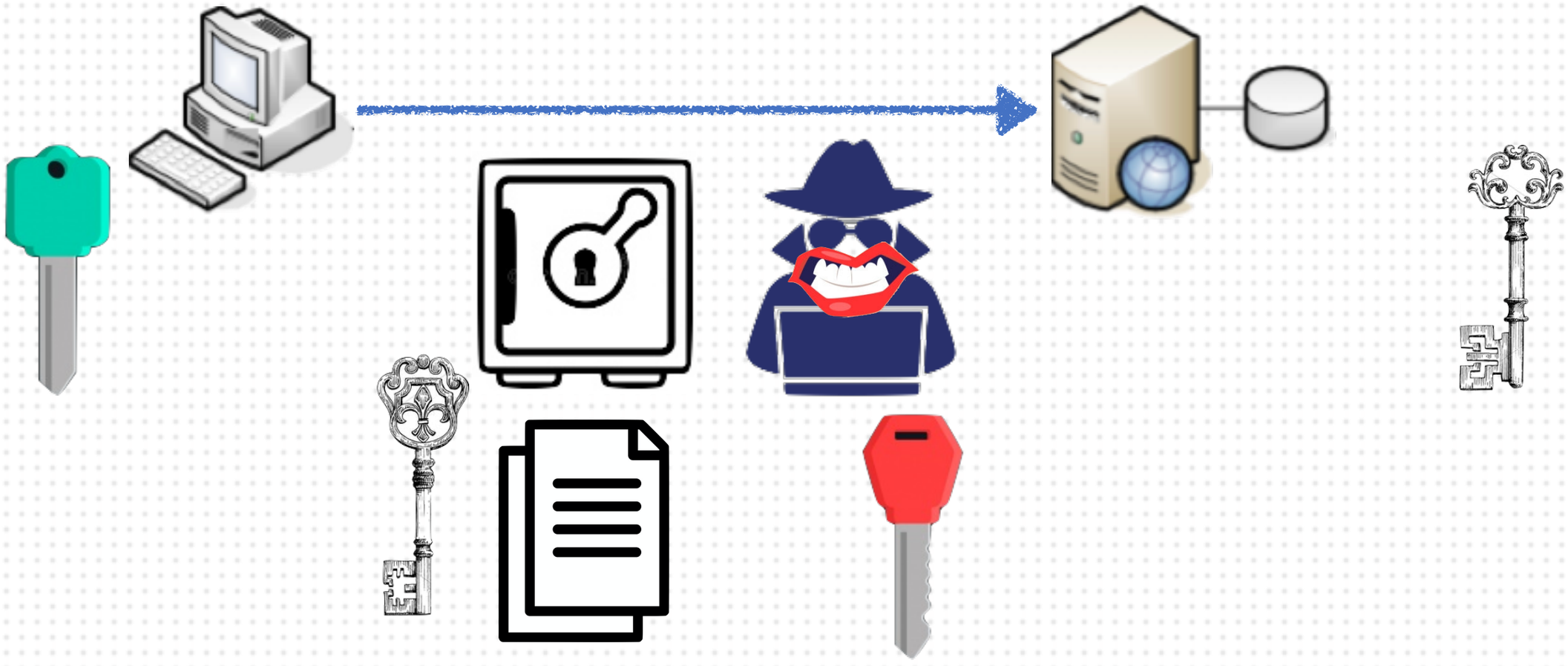




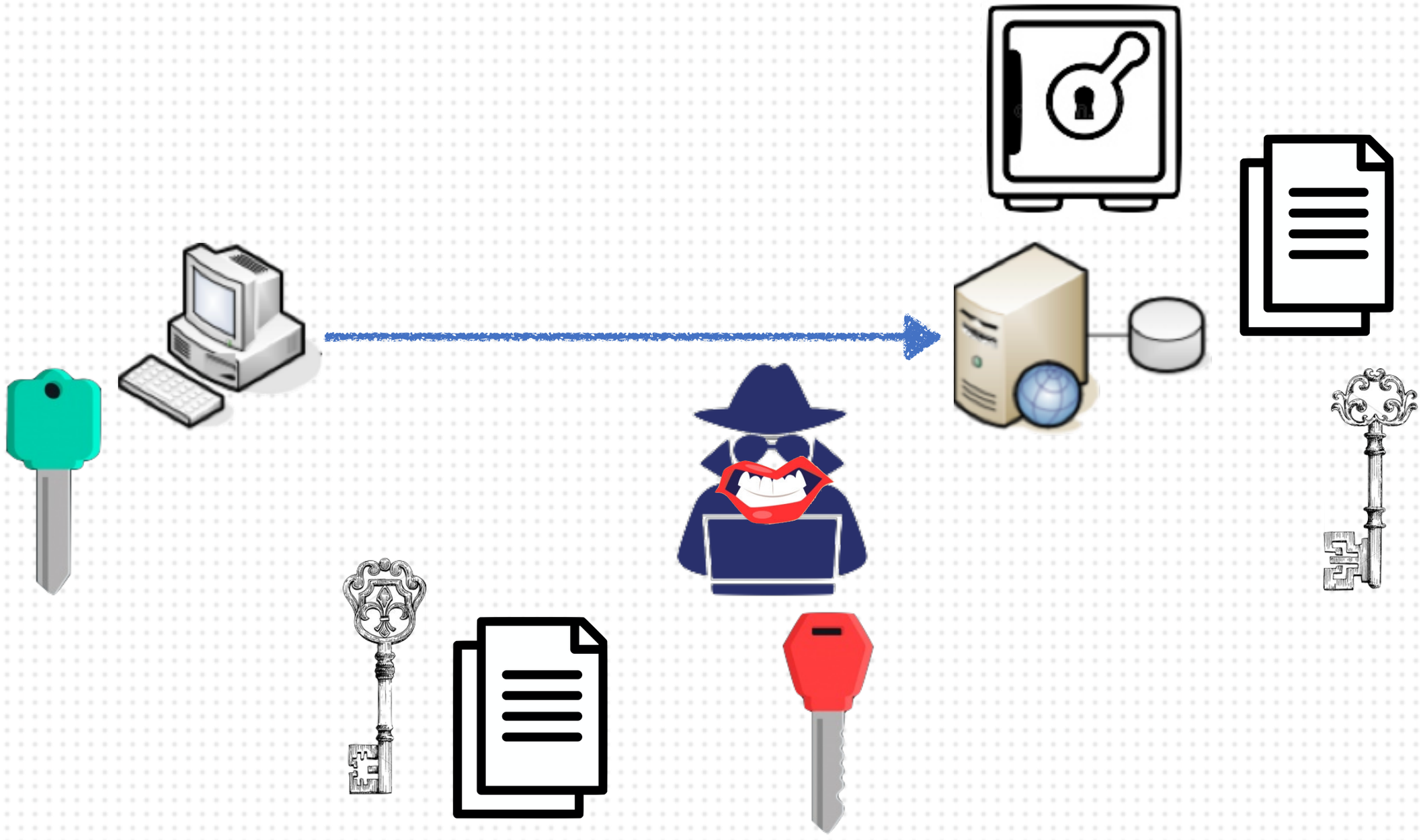
Asymmetric Key - Attack Scenario



Asymmetric Key - Attack Scenario

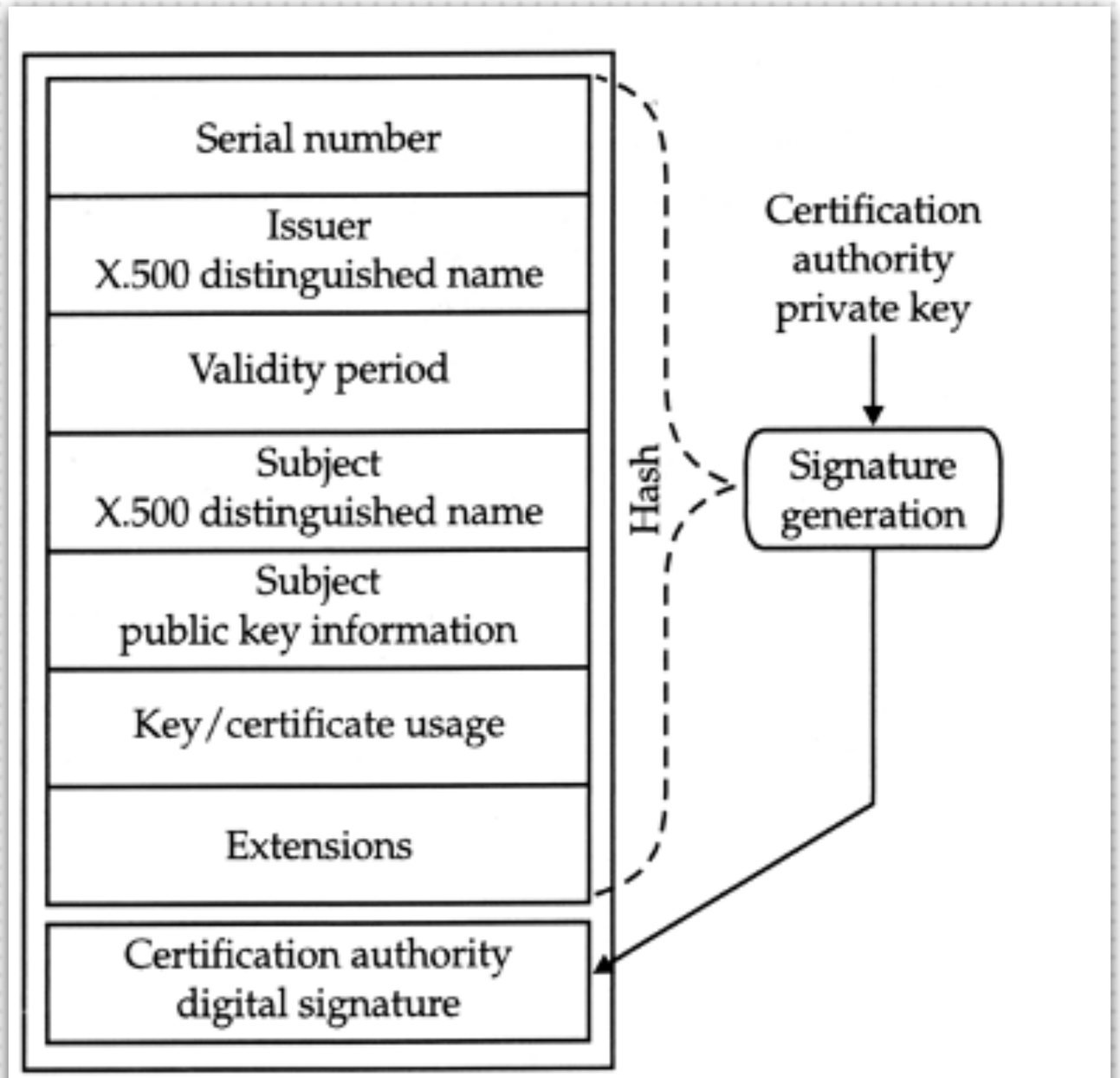


Asymmetric Key - Attack Scenario



Asymmetric Key - Make It Secure!

- Some known parties have to certify keys
- Certificates of some of the most well-known Certificates Authorities (root CAs) are put in operating systems by OS providers.
- So, the starting point of validation mechanism is embedded in OSs.



How Digital Certificates Work

- The CA signs the server's public key with their private key and then gives that signature to the server.
- Servers send both their public keys and the signatures to clients.
- Clients already have the public keys of all CA's they trust. Clients use those public keys to verify that the signatures on the public keys of any servers they communicate with are valid.

Only for Transforming Credential Info?

- Using root CA, we are able to transfer username/ password in a secure way.
- We have to do the same for any kind of information that we want to keep secret
- But, what about secure communication mechanism?
  - This idea is a basis for HTTPS connection (all secure connections)
  - Secure connection guarantees secure transformation of data and it hides complexities